UNIT 3

Exercise 7 : Domain-specific chatbot using LangChain and a vector database

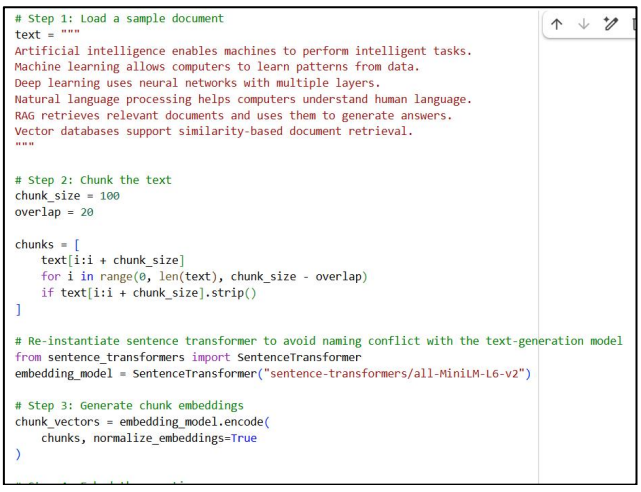
Aim

To generate numerical embeddings for text and identify semantically similar sentences using embedding similarity.

Algorithm

- Start the program.
- Load a pretrained sentence-embedding model.
- Define a collection of text documents.
- Convert the documents into numerical vectors.
- Convert the user's query into an embedding.
- Calculate similarity scores between the query and documents.
- Display the most similar document.
- Stop.

Python Code and Output



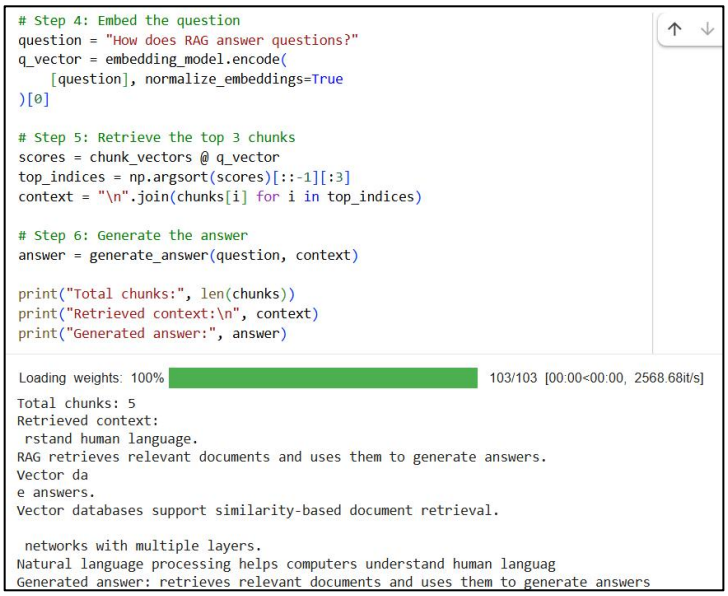
```
# Step 1: Load a sample document
text = """
Artificial intelligence enables machines to perform intelligent tasks.
Machine learning allows computers to learn patterns from data.
Deep learning uses neural networks with multiple layers.
Natural language processing helps computers understand human language.
RAG retrieves relevant documents and uses them to generate answers.
Vector databases support similarity-based document retrieval.
"""

# Step 2: Chunk the text
chunk_size = 100
overlap = 20

chunks = [
    text[i:i + chunk_size]
    for i in range(0, len(text), chunk_size - overlap)
    if text[i:i + chunk_size].strip()
]

# Re-instantiate sentence transformer to avoid naming conflict with the text-generation model
from sentence_transformers import SentenceTransformer
embedding_model = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")

# Step 3: Generate chunk embeddings
chunk_vectors = embedding_model.encode(
    chunks, normalize_embeddings=True
)
```



```
# Step 4: Embed the question
question = "How does RAG answer questions?"
q_vector = embedding_model.encode(
    [question], normalize_embeddings=True
)[0]

# Step 5: Retrieve the top 3 chunks
scores = chunk_vectors @ q_vector
top_indices = np.argsort(scores)[::-1][:3]
context = "\n".join(chunks[i] for i in top_indices)

# Step 6: Generate the answer
answer = generate_answer(question, context)

print("Total chunks:", len(chunks))
print("Retrieved context:\n", context)
print("Generated answer:", answer)
```

```
Loading weights: 100% 103/103 [00:00<00:00, 2568.68it/s]
Total chunks: 5
Retrieved context:
 rstand human language.
RAG retrieves relevant documents and uses them to generate answers.
Vector da
e answers.
Vector databases support similarity-based document retrieval.

 networks with multiple layers.
Natural language processing helps computers understand human languag
Generated answer: retrieves relevant documents and uses them to generate answers
```

Result

Text embeddings were generated successfully, and semantic similarity search identified the most relevant document.